EventCtf: Capture the Flag

Example 1 (v1.1.5) vs Example 2 (v2.0.1)

Performance & code quality comparison for a high-population Rust server

Generated 04 November 2026

Verdict: run Example 2 (v2.0.1).

- It fixes almost every real bug in v1.1.5, including the two that could break a live event, and closes most of the teleport loophole.
- It is only leaner during the spawn search. During a live event it costs exactly the same as v1.1.5: the two biggest per-second costs were not touched.
- It adds six small issues of its own (R-01 to R-06). Each is a short fix.

Executive Summary

Both files are the same event. A flag spawns at a random spot, and the carrier follows a chain of checkpoints to a monument. While they carry it, vehicles, teleports and safe zones are blocked. Example 2 (v2.0.1 by Nathan, 3,231 lines) is a rewrite of Example 1 (v1.1.5 by Adem, 2,369 lines). It keeps the same event loop and config layout.

Example 2 is the better plugin. It makes 19 improvements over Example 1 (page 2). Two of them fix bugs that could break a live event. In v1.1.5, a destroyed or despawned flag left the event running with nothing to capture, and a checkpoint could count twice. v2.0.1 blocks teleports through the `CanTeleport` API instead of a four-command chat list. It also adds a permission, survives a broken config file and cleans up after a crash. Its spawn search only samples valid land and creates no garbage.

It is not faster during a live event. The two biggest recurring costs are identical in both versions. Every second, the carrier is briefly given the admin flag so an on-screen marker can be drawn, which sends full player updates to everyone nearby. Also every second, both map markers are resent to every connected player, even when the flag hasn't moved. With 300 players online and 60 near the flag, that is about 900 marker messages and at least 120 full player updates per second. Fixing these two (EC-01 and EC-02) is the biggest win available on a high-pop server, and neither fix is large.

It also adds six issues of its own (R-01 to R-06). Fix two of them first. A pooled dictionary is passed to EventManager and recycled as soon as the call returns. The map-marker protection uses a hook signature that Oxide stopped calling after 31 December 2023. Two gameplay holes remain in both versions: the carrier can log off with the flag (EC-03), and can ride standing on boats, trains, balloons and the cargo ship (EC-04).

Scorecard

Area	v1.1.5 (Example 1)	v2.0.1 (Example 2)	Better
Cost with no event running	Hooks off, close to zero	Hooks off, close to zero; keeps a 4 MB spawn map in memory	Tie
Plugin load	Trivial	Sweeps every player and sleeper; builds the spawn map over ~256 frames	v1.1.5
Spawn search	Random circle: ~16% of samples off the map, plus ocean; allocates on every check	Pre-filtered land map; no allocations in the checks	v2.0.1
Per-second cost during an event	Admin-flag screen marker + 3 marker broadcasts	Identical	Tie
Busy hooks during an event	Cheap early exits	Same, plus O(1) command lookup and one OnEntityEnter	v2.0.1
Flag destroyed or despawned	Not detected; event runs to timeout	Detected within 1 s; despawn cancelled on every drop	v2.0.1

Area	v1.1.5 (Example 1)	v2.0.1 (Example 2)	Better
Checkpoints	Can count twice; can throw after a win	Guarded	v2.0.1
Teleport blocking	4 chat commands	Same + CanTeleport API	v2.0.1
Vehicle blocking	Seats + scrap heli platform	Same	Tie
Carrier logs off	Not handled	Not handled	Tie
Safe-zone kick	Spheres only; ignores scale	Spheres only; uses world size	v2.0.1
Event marker protection	Works (current hook signature)	Retired signature; never runs	v1.1.5
Crash leftovers	Vending marker can persist	Markers not saved; startup sweep	v2.0.1
Config handling	Bad JSON breaks the load	Backup, safe defaults, migration	v2.0.1
Admin access	player.IsAdmin	eventctf.admin permission	v2.0.1
Discord logs	Wrong duration; posts on every reload	Correct	v2.0.1
Data passed to EventManager	Fresh dictionary	Pooled dictionary, recycled after the call	v1.1.5

Result: v2.0.1 better in 10 areas, v1.1.5 in 3, tie in 4. All three areas where v1.1.5 is better are cheap to fix in v2.0.1.

What Example 2 (v2.0.1) does better

Line numbers refer to each file as uploaded. "v1 L798" means line 798 of Example 1.

#	Area	Example 1 (v1.1.5)	Example 2 (v2.0.1)
1	Flag destroyed or despawned	`_item == null` can never be true for a removed item (v1 L798), so the event keeps running with no flag	`IsLiveFlag()` checks `Item.IsValid()` and the stored item ID every tick and ends the event (L2278-2285, L2599)
2	Re-dropped flag despawning	Despawn timer cancelled only on the first spawn (v1 L672). A flag dropped on death can despawn	Cancelled on every dropped copy, every tick (L2640-2644)
3	Nearby alerts	Loops over the whole shared 64-slot AI buffer, so players left over from NPC scans anywhere on the map get alerts (v1 L867, L916)	Loops only over the slots just filled (L2683, L2747)
4	Checkpoints	No 'already used' guard. A checkpoint can count twice, or throw after a win (v1 L1122-1130)	`_consumed` guard and null checks (L1449-1461)
5	Vending map marker	Saved with the map, so it can stay after a crash (v1 L1169)	`enableSaving = false` before `Spawn()` (L1518)
6	Teleports	Only /home, /tp, /tpr, /tpa typed in chat (v1 L208-220)	Adds `CanTeleport`, which blocks the teleport itself in any plugin that calls it, including /town, /outpost and menu buttons (L1755-1767)
7	Blocked-command lookup	Culture-sensitive linear scan with a closure (v1 L213)	O(1) case-insensitive HashSet (L419-433, L1776)
8	Discord logs	Always reports the configured length. Posts 'no winner' on every reload and failed spawn (v1 L984, L265, L71)	Reports the time that actually passed. No post on unload or failed spawn (L2292-2296, L1614, L2381)
9	Config file	Bad JSON breaks the load. The file is rewritten on every load (v1 L1546-1551)	Timestamped backup, safe defaults, migration and filling of missing sections. Saves only when something changed (L84-230, L288-313, L376-410)
10	Admin commands	`player.IsAdmin`	`eventctf.admin` permission. Players using the console commands are checked too (L1581, L1799-1811)
11	Stale markers and crash leftovers	Teammates who went offline keep their CTF marker	Tracks every player given a marker and cleans them up, online or not. Startup and connect sweep (L2266, L2542-2566, L1922-2010)

#	Area	Example 1 (v1.1.5)	Example 2 (v2.0.1)
12	Coroutines	Run on `ServerMgr` and are stopped by hand	Run on the plugin's own object, destroyed on unload. Every loop checks it still belongs to the current event (L1343-1349, L2315, L2597)
13	Spawn sampling	Random point in a circle 1.2x the map width. About 16% of samples land off the map, and more in the ocean (v1 L1285-1290)	`SpawnDistribution` over a pre-filtered topology map: every sample is on valid land (L60-68, L1845-1880, L2332)
14	Spawn checks (garbage)	New HashSet, new collider array and 1-2 new strings per collider on every check (v1 L1349-1378)	Shared 512-slot buffer and a type check for safe zones. No allocations (L2899, L3024-3058)
15	Waypoints and spawn terrain	A checkpoint could land on a riverbed or lake bottom	Rejects water deeper than 0.25 m. The flag spawn also excludes lakes and cliffsides (L2996-3007, L66-67)
16	Safe-zone kick	Uses the collider's local radius and ignores its scale (v1 L311)	Uses the zone's real size in the world (L2014-2015)
17	Config lookups	Linear search with a closure for every message (v1 L1425)	Dictionaries built once (L435-479, L3191-3198)
18	/ctfnewendpoint	Uses the nearest monument centre; can throw on custom maps (v1 L433)	Requires standing inside the monument's bounds. Saves 3 decimals in an invariant format (L2156-2248)
19	Messages	'Node Event ends in…' left over from another plugin (v1 L1464)	'Capture The Flag event ends in…' (L1308)

Performance: where v2.0.1 is faster, and where it isn't

The plugin's cost comes in five phases. v2.0.1 is clearly better in one (the spawn search), slightly better in two, and identical in the one that runs for the whole event.

Phase	v1.1.5	v2.0.1	Verdict
No event running	Close to zero. All event hooks unsubscribed	Close to zero. Only `OnPlayerConnected` stays subscribed (cheap). Keeps ~4 MB in memory	Tie
Plugin load	Trivial	Synchronous sweep of every player and sleeper. Spawn map built over 256 frames, 16,384 lookups per frame	v1.1.5
Spawn search (event start)	~16% of samples off the map, plus ocean. Each candidate allocates a HashSet, an array and many strings	Every sample on valid land. Physics checks allocate nothing	v2.0.1
Live event, every second	3 map-marker messages to every player + at least 2 full player updates to everyone near the carrier	Identical, line for line	Tie
Busy hooks during the event	Cheap early exits on item ID / player checks	Same. One OnEntityEnter instead of two overloads; O(1) command lookup	v2.0.1
Broadcast messages	Formatted per player; closure + linear search per player	Formatted per player; dictionary lookup per player	v2.0.1

What the per-second cost looks like at high pop

Example: 300 players online, 60 of them within network range of the flag carrier. These numbers are the same for both versions.

Source	Per tick	Recipients	Per second
Map markers (vending + ring)	3 network messages	Every connected player (300)	~900 messages
Admin-flag screen marker	2x `SendNetworkUpdateImmediate()`	Everyone near the carrier (60)	at least 120 full player updates

Both run every second for the whole event, which is an hour by default, and both mostly send nothing new. The flag usually sits still or moves a few metres, and the marker label only changes once a minute. A full player update is one of the larger messages Rust sends, and here it goes to the busiest spot on the map. EC-01 cuts the player updates by about 75% (or removes them), and EC-02 cuts marker traffic by over 99% while the flag is still, without changing what players see.

Why the spawn-search change matters

In v1.1.5, each candidate spot runs a 20 m physics query with no layer filter, which allocates a new array of every collider found. Then it creates a new HashSet, reads every collider's name (a new string each time) and lower-cases it (another string), and builds a closure. All of this happens up to 10 times per frame, plus up to 10 smaller checks per candidate for the waypoints. That garbage has to be collected later, and collection on a large high-pop heap is expensive. v2.0.1 removes all of it, and its sampler never wastes attempts on the ocean or off-map points. The remaining per-entity work is covered in EC-06.

Map sampling and spawn map numbers

Item	Value
v1.1.5 random-point sampler	Circle of radius 0.6 x map size; 15.9% of points fall outside the map (simulated)
v2.0.1 spawn map size	ClosestPowerOfTwo(map size / 2) squared: 2048 x 2048 = 4 MB for maps 3,500-6,000; 1 MB for 3,000
v2.0.1 spawn map build	8 rows per frame: 256 frames x 16,384 topology lookups (2048 map)
Spawn attempts per frame	10 in both versions

What Example 2 (v2.0.1) does worse

These issues are new in v2.0.1. None of them is in v1.1.5.

R-01 **MEDIUM (risk)** **Pooled dictionary passed to EventManager, then recycled**

Where: v2.0.1 L2824-2833 (v1.1.5 L979-981 was safe)

`OnOwnerWin` takes a `Dictionary` from the pool and passes it to the `OnEventEnd` hook. It returns the dictionary to the pool as soon as the call comes back. If EventManager, or any plugin listening to that hook, keeps the reference and reads it later (in a timer, NextTick or a stored list), it will find it empty or holding someone else's data, and the clan win may not be recorded. v1.1.5 passed a fresh dictionary, which is always safe. Pooling here saves one tiny allocation per event, so it isn't worth the risk.

```
// v2.0.1 L2824-2833
Dictionary<string, bool> clansForSending = Pool.Get<Dictionary<string, bool>>();
try   { clansForSending.Add(clanTag, true); Interface.CallHook("OnEventEnd", clansForSending); }
finally { Pool.FreeUnmanaged(ref clansForSending); }   // cleared while EventManager may still hold it
```

Fix

```
Interface.CallHook("OnEventEnd", new Dictionary<string, bool> { [clanTag] = true });
```

Rule of thumb: never return a pooled object to the pool after giving it to another plugin.

R-02 **LOW** **Map-marker protection uses a hook signature Oxide no longer calls**

Where: v2.0.1 L1744-1753

v2.0.1 changed the hook to `OnMapMarkerRemove(BasePlayer player, MapNote note)`. Oxide deprecated that form in 2023 and stopped calling it after 31 December 2023. The current hook is `OnMapMarkerRemove(BasePlayer player, List<MapNote> mapMarker, int index)`, which v1.1.5 uses. In v2.0.1 the method never runs, so players can delete the event's destination marker. The gameplay impact is small, but the feature has silently stopped working, and the fix is one line.

Fix (the v1.1.5 signature)

```
private object OnMapMarkerRemove(BasePlayer player, List<MapNote> mapMarker, int index)
{
    if (_eventController == null || player == null || mapMarker == null
        || index < 0 || index >= mapMarker.Count)
        return null;

    return _eventController.IsEventMapMarker(mapMarker[index]) ? (object)true : null;
}
```

R-03 **LOW** **enableSaving = false set after the flag has already spawned**

Where: v2.0.1 L2433, L2642

`SpawnFlag` and `UpdateFlagOwner` set `enableSaving = false` on the dropped flag after `Item.Drop()` has already spawned it. Rust adds an entity to its save list when it spawns, and as far as I know only `BaseEntity.EnableSaving(false)` takes it back out; assigning the field afterwards doesn't. If that's right, the flag is still written to the save. After a crash it comes back as a normal item on the ground, and the startup sweep doesn't catch it because it only cleans player inventories. The map markers and spheres set the field before `Spawn()`, which is correct.

Fix

```
DroppedItem droppedItem = _item.Drop(position, Vector3.up) as DroppedItem;
droppedItem.EnableSaving(false);                // updates the save list, not just the field
droppedItem.CancelInvoke(droppedItem.IdleDestroy);
```

R-04 LOW Startup and connect sweeps are too broad and run all at once

Where: v2.0.1 L1922-2010

At startup the plugin walks every online player and every sleeper. It removes every `twitchrivalsflag` item and every map note labelled "CTF", and does the same for each player who connects while no event is running. This has two side effects:

- Any flag or "CTF" marker that didn't come from this plugin is deleted too. A player's own marker named "CTF" disappears every time they log in.
- Late in a wipe the sleeper list can hold thousands of players. The sweep does them all in a single frame, so reloading the plugin causes a one-off lag spike.

Fix

- Tag the event flag when it's created (`_item.name = "CTF Event Flag";`) and only remove items with that name.
- Save `_markedPlayers` to a small data file and clean only those players, not everyone with a "CTF" note.
- Or spread the sleeper sweep over frames on the existing `CoroutineManager`, for example 200 players per frame.

R-05 LOW Spawn map held in memory for the plugin's whole lifetime

Where: v2.0.1 L1845-1880

`BuildSpawnMap` allocates a byte map of `ClosestPowerOfTwo(World.Size / 2)` squared: 2048 x 2048 = 4 MB for any map from 3,500 to 6,000. It then keeps that map, plus the quadtree `SpawnDistribution` builds on top of it, for as long as the plugin is loaded. Building it takes 256 frames with 16,384 topology lookups per frame. It's a one-off cost and acceptable, but 1024 x 1024 (1 MB, about 4.4 m cells on a 4,500 map) is plenty for picking a flag spawn.

Fix (optional)

```
int resolution = Mathf.ClosestPowerOfTwo((int)(World.Size * 0.25f));   // 1024 on 3.5k-6k maps
```

R-06 LOW Waypoint placement rules changed; flag point checked twice

Where: v2.0.1 L2340, L2953-3010 (v1.1.5 L1303-1319)

v1.1.5 rejected any waypoint more than 0.5 m above or below bare terrain. v2.0.1 replaced that with Rust's own spawn placement checks plus a water-depth test. The water test is a real improvement: no more checkpoints on a riverbed. But if the engine's placement mask accepts rocks or other world objects, a waypoint can now sit on top of one. Separately, the flag point from `SpawnDistribution.Sample` goes through the same placement checks again. As far as I can tell, `Sample` already runs them, so that is one extra raycast and sphere check per sample.

Fix

```
// IsIntermediatePositionSuitable: keep v2's checks and restore v1's terrain test
if (Mathf.Abs(TerrainMeta.HeightMap.GetHeight(position) - position.y) > 0.5f)
    return false;

// TryPrepareFlagPosition: Sample() already placed the point; only run the blocked-area test
return !IsBlockedPoint(position, _ins._config.FlagDelivery.SpawnBlockedRadius);
```

Still in both versions: what to improve for high pop

These apply to v2.0.1 (the version to keep). Line numbers are v2.0.1 unless marked v1.

EC-01 **HIGH** **Admin-flag screen marker: full player updates every second**

Where: L2456-2478, called every tick at L2660 (v1 L694-716, L852)

To draw the destination marker with `ddraw.text`, `UpdateScreenMarkerForOwner` gives a non-admin carrier the admin flag, sends the text, then removes the flag again. It does this every second, from `UpdateFlagOwner`. Each toggle calls `SendNetworkUpdateImmediate()`, which sends a full player update to every player in range, not just the carrier. With 60 players near the flag, that is at least 120 full player updates per second for the whole event, in the busiest spot on the map. It also keeps running after the carrier disconnects, because their sleeping body still holds the flag. And handing a client the admin flag every second is something some anti-cheat setups treat as suspicious.

```
// v2.0.1 L2463-2478 - runs every second for a non-admin carrier
SwitchMomentRules(_owner, true);    // SetPlayerFlag(IsAdmin, true) + SendNetworkUpdateImmediate()
_owner.SendConsoleCommand("ddraw.text", 1f, Color.white, _currentDestination, text);
SwitchMomentRules(_owner, false);   // SetPlayerFlag(IsAdmin, false) + SendNetworkUpdateImmediate()
```

Fix

- **Best:** turn the screen marker off by default. The destination map note already shows on the player's compass, and the zone sphere appears within 25 m.
- **If you keep it:** draw for 5 s and refresh every 4 s (about 75% fewer updates). Skip offline carriers, build the text once, and force a redraw when the destination changes.

```
private float _nextScreenMarkerAt;
private string _screenText;                       // build once in StartEvent

private void UpdateScreenMarkerForOwner(bool force = false)
{
    if (_owner == null || !_owner.IsConnected || !_ins._config.ScreenMarkerConfig.Enabled)
        return;

    float now = Time.realtimeSinceStartup;
    if (!force && now < _nextScreenMarkerAt)
        return;
    _nextScreenMarkerAt = now + 4f;               // refresh every 4 s ...
    const float duration = 5f;                    // ... and draw for 5 s, so there is no gap

    if (_owner.IsAdmin)
    {
        _owner.SendConsoleCommand("ddraw.text", duration, Color.white, _currentDestination, _screenText);
        return;
    }

    SwitchMomentRules(_owner, true);
    _owner.SendConsoleCommand("ddraw.text", duration, Color.white, _currentDestination, _screenText);
    SwitchMomentRules(_owner, false);
}
// call UpdateScreenMarkerForOwner(true) on pickup and after UpdateDestination()
```

Going further, the temporary admin-flag update could go only to the carrier's own connection instead of everyone in range. Check which per-connection snapshot method your server build exposes before relying on that.

EC-02 **MEDIUM** **Map markers resent to every player every second**

Where: L2607-2608 -> L1542-1566 (v1 L806-807, L1193-1217)

`EventCoroutine` calls `UpdateMapMarker` on every tick. That moves both markers, rebuilds the label (up to three lang lookups and a string.Format), and sends three network messages: vending `SendNetworkUpdate`, ring `SendUpdate`, and ring `SendNetworkUpdate`. Map markers go to every connected player, so with 300 online that's about 900 messages per second, even while the flag lies still on the ground. The label only changes once a minute, because it shows minutes until the final minute.

Fix: only send when something changed

```
private Vector3 _lastPos = new Vector3(float.MaxValue, 0f, 0f);
private string _lastName;

public void UpdateMapMarker(Vector3 position)
{
    string name = GetMarkerName();
    bool moved = (position - _lastPos).sqrMagnitude > 9f;      // moved more than 3 m
    if (!moved && name == _lastName)
        return;                                                // nothing new for clients

    _lastPos = position;
    _lastName = name;

    if (_vendingMarker.IsExists())
    {
        _vendingMarker.transform.position = position;
        _vendingMarker.markerShopName = name;
        _vendingMarker.SendNetworkUpdate();
    }

    if (moved && _radiusMarker.IsExists())
    {
        _radiusMarker.transform.position = position;
        _radiusMarker.SendNetworkUpdate();
        // SendUpdate() resends colour and radius. Try without it; restore it if the ring
        // stops following the flag.
    }
}
```

A flag on the ground drops from 3 messages a second to 1 a minute. A carrier on the move still updates every second.

EC-03 HIGH Carrier can log off with the flag

Where: no handler in either version

Neither version handles a carrier who disconnects. The flag stays in the sleeping body's wear slot and the map marker keeps pointing at it. The event then only ends when it times out, unless someone gets to the sleeper. A carrier who logs off inside their own base stalls the event for everyone. Because of EC-01, the plugin also keeps toggling the admin flag on that sleeper every second.

Fix: drop the flag on disconnect

```
// add "OnPlayerDisconnected" to _subscribeMethods so it only runs during an event
private void OnPlayerDisconnected(BasePlayer player, string reason)
{
    if (_eventController != null && _eventController.IsEventPlayer(player.userID))
        _eventController.DropFlag(player);
}

// EventController
public void DropFlag(BasePlayer player)
{
    if (!IsLiveFlag())
        return;

    _item.Drop(player.transform.position + Vector3.up * 0.5f, Vector3.up);
    // On the next tick UpdateFlagOwner sees the dropped flag, cancels its despawn timer
    // and runs OnPlayerLostFlag (team and nearby alerts), exactly as for a death.
}
```

EC-04 MEDIUM Vehicle block only covers seats and the scrap heli

Where: L1694-1742 (v1 L144-176)

`CanMountEntity` blocks seats, and `OnEntityEnter` only handles the scrap heli's rear platform. The carrier can still ride standing on a tugboat deck, a train wagon, a hot air balloon basket or the cargo ship. None of those need a seat.

Fix: check the carrier's parent in the 1-second tick

This checks one player once a second, so the cost is negligible. Sending the carrier back to where they last stood on foot also replaces the scrap-heli "5 m behind" teleport, which can drop them from height (EC-09).

```
private Vector3 _lastFootPosition;

// in UpdateFlagOwner, after _owner = player;
BaseEntity parent = _owner.GetParentEntity();
if (parent == null)
{
    if (_owner.IsOnGround())
        _lastFootPosition = _owner.transform.position;      // remember a known-good spot
}
else if (parent is BaseVehicle || parent is HotAirBalloon || parent is CargoShip)
{
    NotifyManager.SendMessageToPlayer(_owner, "Flag_Vehicle_Blocked");
    BasePlayer carrier = _owner;
    Vector3 back = _lastFootPosition;
    _ins.NextTick(() => { if (carrier != null && !carrier.IsDead()) carrier.Teleport(back); });
}
```

EC-05 MEDIUM Destination marker skipped when the player has no marker list

Where: L2490 (v1 L728)

`CreateMapMarkerForPlayer` returns early if `player.State.pointsOfInterest` is null. As far as I know, Rust only creates that list the first time a player places a marker. If so, a player who has never placed one gets no destination marker at all. Test it with a fresh account.

Fix

```
if (player == null || player.State == null || !_ins._config.TargetMapMarkerConfig.Enabled)
    return;

if (player.State.pointsOfInterest == null)
    player.State.pointsOfInterest = Pool.Get<List<MapNote>>();
```

EC-06 LOW Spawn-block check still does work for every entity it finds

Where: L3024-3058 (v1 L1349-1378)

`IsBlockedPoint` no longer allocates, which is the big win over v1.1.5. But it still collects every collider in range (up to 512) with no layer filter. For each one it walks up the parents looking for a `TriggerSafeZone`, and it calls `GetBuildingPrivilege()` (itself a physics query) for every entity, trees included. This only runs during the spawn search, but the search does up to 10 attempts per frame.

Fix: one masked query, plus a direct safe-zone test

```
private const int StructureMask = (1 << (int)Layer.Construction) | (1 << (int)Layer.Deployed);

private static bool IsBlockedPoint(Vector3 position, float radius)
{
    if (radius <= 0f)
        return false;

    // Any building block or deployable in range: one query, no per-entity work.
    if (Physics.CheckSphere(position, radius, StructureMask, QueryTriggerInteraction.Ignore))
        return true;

    // A map has only a handful of safe zones: test their bounds directly.
    float r2 = radius * radius;
    foreach (TriggerSafeZone zone in TriggerSafeZone.allSafeZones)
    {
        Collider col = zone != null ? zone.triggerCollider : null;
        if (col != null && col.bounds.SqrDistance(position) <= r2)
            return true;
    }
    return false;
}
```

This blocks spots near any player structure, not only ones covered by a tool cupboard, which is usually what you want for a flag spawn.

EC-07 LOW Countdown runs slow when the server is under load

Where: L2595, L2610, L2625 (v1 L809, L824, L827)

Each tick waits at least one second, then subtracts exactly one. The wait always overshoots by part of a frame, so the event runs longer than configured. At 20 server FPS, a one-hour event runs about 90 seconds over, and the end-of-event warnings drift the same way.

Fix: count down against the clock

```
private float _endsAt;      // in StartEvent: _endsAt = Time.realtimeSinceStartup + EventDuration;

// in EventCoroutine, instead of _remainTime -= tick:
int remain = Mathf.Max(0, Mathf.CeilToInt(_endsAt - Time.realtimeSinceStartup));
foreach (int t in _ins._config.EndEventNotifyTimes)
    if (_remainTime > t && remain <= t)              // crossed a warning time since last tick
        NotifyManager.SendMessageToAll("Event_TimeLeft", _ins._config.ChatPrefix, t);
_remainTime = remain;
if (_remainTime <= 0) { _ins.StopEvent(); yield break; }
```

EC-08 LOW Broadcasts are built once per player

Where: L3151-3189 (v1 L1383-1421)

`SendMessageToAll` builds the message separately for every online player: a new argument array, the time strings, a lang lookup, string.Format, and its own chat command string. With 300 online that's 300 copies of the same work per broadcast. Broadcasts are rare, so this matters less than EC-01 and EC-02, but it's an easy win: build the message once per language.

Fix

```
public static void SendMessageToAll(string langKey, params object[] args)
{
    RedefinedMessageConfig redefined = GetRedefinedMessageConfig(langKey);
    if (redefined != null && !redefined.IsEnable)
        return;
    BaseMessageConfig target = redefined ?? (BaseMessageConfig)_ins._config.NotifyConfig;

    Dictionary<string, string> byLanguage = Pool.Get<Dictionary<string, string>>();
    try
    {
        foreach (BasePlayer player in BasePlayer.activePlayerList)
        {
            if (player == null) continue;
            string language = _ins.lang.GetLanguage(player.UserIDString);
            if (!byLanguage.TryGetValue(language, out string message))
                byLanguage[language] = message = Format(player, langKey, args);  // today's logic
            SendMessage(target, player, message);
        }
    }
    finally { Pool.FreeUnmanaged(ref byLanguage); }
}
```

EC-09 LOW Safe-zone and scrap-heli edge cases

Where: L1653-1742, L2012-2032

- Only safe zones with a `SphereCollider` are handled. A box- or mesh-shaped zone can't eject the carrier (L1685-1687, L1736-1738).
- Picking up the flag on a scrap heli's platform returns before the safe-zone check runs (L1670-1677).
- The scrap-heli teleport puts the carrier 5 m behind and 1 m above the heli with no ground check. If the heli is flying, the carrier falls (L1673, L1729).

Fix

```
// Works for any collider shape; exact for spheres.
Bounds b = zoneCollider.bounds;
Vector3 dir = player.transform.position - b.center;
dir.y = 0f;
dir = dir.sqrMagnitude < 0.001f ? Vector3.forward : dir.normalized;
float reach = zoneCollider is SphereCollider
    ? b.extents.x + 2f
    : new Vector2(b.extents.x, b.extents.z).magnitude + 2f;
Vector3 exit = PositionDefiner.GetGroundPosition(b.center + dir * reach) + Vector3.up * 0.25f;
```

Run the safe-zone check before the scrap-heli branch returns. Send a carrier who is on a heli to their last on-foot position (EC-04) instead of 5 m behind it.

EC-10 LOW Nearby alerts capped at 64 players

Where: L2679, L2743

The nearby pickup and drop alerts still borrow `AIBrainSenses.playerQueryResults`, a 64-slot buffer that belongs to the NPC AI. v2.0.1 fixed the stale-entry bug, but in a busy event more than 64 players can be within 100 m, and nobody past the 64th gets an alert.

Fix

```
private static readonly BasePlayer[] NearbyPlayers = new BasePlayer[256];   // the plugin's own buffer
int count = BaseEntity.Query.Server.GetPlayersInSphereFast(_flagPosition, radius, NearbyPlayers);
```

EC-11 LOW Small leftovers

- `Flag_Picked_Up` fills `{0}` with the chat prefix, so the carrier sees "[CTF] | Picked up the flag" instead of their name. Pass the player's name, or change the lang text (L1293, L2769).
- `/ctffinish` awards whoever held the flag at the last tick. If it was dropped within that second, the wrong player can win. Call `UpdateFlagOwner()` first (L2777-2785).
- `OnPlayerCommand` only sees chat commands. With `CanTeleport` in place this matters less, but plugins that teleport players directly (some warp and menu plugins) still get through (L1771-1783).
- `CanTeleport` sends its own message and returns an empty string. Teleport plugins that print the returned string will add a blank line. Return the message text instead and drop the extra send (L1760-1763).

Priority fix roadmap (for v2.0.1)

#	ID	Action	Effort	Impact
1	R-01	Pass a fresh dictionary to OnEventEnd	2 min	Interop safety
2	R-02	Restore the current OnMapMarkerRemove signature	5 min	Regression fix
3	EC-05	Create pointsOfInterest when it's null	5 min	Markers for every player
4	EC-02	Only resend map markers when they moved or the label changed	30 min	Up to 99% fewer broadcasts
5	EC-01	Screen marker off by default, or 4 s refresh + skip offline carriers	30 min	75-100% fewer player updates
6	EC-03	Drop the flag when the carrier disconnects	30 min	Closes the log-off stall
7	EC-04	Parent check in the tick; send riders back to their last on-foot spot	1 hr	Closes vehicle rides
8	R-03	Use EnableSaving(false) on the dropped flag	5 min	Crash safety
9	EC-07	Count down against the real clock	30 min	Correct event length
10	EC-06	One masked query + direct safe-zone test in IsBlockedPoint	1 hr	Cheaper spawn search
11	EC-08	Build broadcasts once per language	1 hr	Fewer allocations
12	EC-09	Any-shape safe zones; safe-zone check before the heli branch	1 hr	Closes edge exploits
13	R-04	Tag the flag; sweep only tagged items and marked players	1-2 hrs	No collateral deletes
14	EC-10, EC-11, R-05, R-06	Own alert buffer; text, /ctffinish and CanTeleport fixes; smaller spawn map; waypoint terrain check	1-2 hrs	Cleanup

Check these on a test server

I couldn't confirm these without the server's game files. Each one decides whether a finding applies to your build.

Finding	What to check
EC-05	Does a fresh account that has never placed a map marker get the CTF destination marker?
R-03	With the flag on the ground, run a save and restart: does the flag come back? (Use `EnableSaving(false)` either way.)
R-01	Does EventManager read the OnEventEnd dictionary after the hook returns (timer, NextTick, stored list)?
EC-02	Does the ring marker follow the flag without `SendUpdate()`?
Teleports	Does your teleport plugin call `CanTeleport`? NTeleportation does. Which plugin calls `ShouldInterrupt`? If none, remove it.
R-06	Can waypoints now land on top of rocks or other world objects?
EC-06	Does `TriggerSafeZone.allSafeZones` exist on your build? It's the static list of safe zones the fix loops over.
EC-01	Do your players see the CTF map note on their compass? If so, the screen marker can go.

References

- Oxide deprecation of OnMapMarkerRemove (old form stopped working 31 December 2023; replacement `OnMapMarkerRemove(BasePlayer player, List<MapNote> mapMarker, int index)`): umod.org/community/rust/51596
- ItemContainer.CanAcceptItem now returns ItemContainer.CanAcceptResult (August 2025), which both versions already use: umod.org/community/npc-drop-gun/56685
- Sampling figure (15.9% off-map) from a 2-million-point simulation of v1.1.5's `GetRandomMapPoint`. Spawn map sizes computed from v2.0.1's `BuildSpawnMap`.